```
/**   多项式（Poly, with MontgomeryModInt32）
 *    2023-08-11: https://ac.nowcoder.com/acm/contest/view-submission?
submissionId=63381475&returnHomeType=1&uid=815516497
 *    *    感谢菜菜园子群友提供
**/
template <std::uint32_t P> struct MontgomeryModInt32 {
public:
    using i32 = std::int32_t;
    using u32 = std::uint32_t;
    using i64 = std::int64_t;
    using u64 = std::uint64_t;

private:
    u32 v;

    static constexpr u32 get_r() {
        u32 iv = P;

        for (u32 i = 0; i != 4; ++i)
            iv *= 2U - P * iv;

        return -iv;
    }

    static constexpr u32 r = get_r(), r2 = -u64(P) % P;

    static_assert((P & 1) == 1);
    static_assert(-r * P == 1);
    static_assert(P < (1 << 30));

public:
    static constexpr u32 pow_mod(u32 x, u64 y) {
        if ((y %= P - 1) < 0)
            y += P - 1;

        u32 res = 1;

        for (; y != 0; y >>= 1, x = u64(x) * x % P)
            if (y & 1)
                res = u64(res) * x % P;

        return res;
    }

    static constexpr u32 get_pr() {
        u32 tmp[32] = {}, cnt = 0;
        const u64 phi = P - 1;
        u64 m = phi;

        for (u64 i = 2; i * i <= m; ++i) {
            if (m % i == 0) {
```

```
            tmp[cnt++] = i;

            while (m % i == 0)
                m /= i;
        }
    }

    if (m > 1)
        tmp[cnt++] = m;

    for (u64 res = 2; res <= phi; ++res) {
        bool flag = true;

        for (u32 i = 0; i != cnt && flag; ++i)
            flag &= pow_mod(res, phi / tmp[i]) != 1;

        if (flag)
            return res;
    }

    return 0;
}

MontgomeryModInt32() = default;
~MontgomeryModInt32() = default;
constexpr MontgomeryModInt32(u32 v) : v(reduce(u64(v) * r2)) {}
constexpr MontgomeryModInt32(const MontgomeryModInt32 &rhs) : v(rhs.v) {}
static constexpr u32 reduce(u64 x) {
    return x + (u64(u32(x) * r) * P) >> 32;
}
static constexpr u32 norm(u32 x) {
    return x - (P & -(x >= P));
}
constexpr u32 get() const {
    u32 res = reduce(v) - P;
    return res + (P & -(res >> 31));
}
explicit constexpr operator u32() const {
    return get();
}
explicit constexpr operator i32() const {
    return i32(get());
}
constexpr MontgomeryModInt32 &operator=(const MontgomeryModInt32 &rhs) {
    return v = rhs.v, *this;
}
constexpr MontgomeryModInt32 operator-() const {
    MontgomeryModInt32 res;
    return res.v = (P << 1 & -(v != 0)) - v, res;
}
constexpr MontgomeryModInt32 inv() const {
    return pow(-1);
```

```
    }
    constexpr MontgomeryModInt32 &operator+=(const MontgomeryModInt32 &rhs) {
        return v += rhs.v - (P << 1), v += P << 1 & -(v >> 31), *this;
    }
    constexpr MontgomeryModInt32 &operator-=(const MontgomeryModInt32 &rhs) {
        return v -= rhs.v, v += P << 1 & -(v >> 31), *this;
    }
    constexpr MontgomeryModInt32 &operator*=(const MontgomeryModInt32 &rhs) {
        return v = reduce(u64(v) * rhs.v), *this;
    }
    constexpr MontgomeryModInt32 &operator/=(const MontgomeryModInt32 &rhs) {
        return this->operator*=(rhs.inv());
    }
    friend MontgomeryModInt32 operator+(const MontgomeryModInt32 &lhs,
                                        const MontgomeryModInt32 &rhs) {
        return MontgomeryModInt32(lhs) += rhs;
    }
    friend MontgomeryModInt32 operator-(const MontgomeryModInt32 &lhs,
                                        const MontgomeryModInt32 &rhs) {
        return MontgomeryModInt32(lhs) -= rhs;
    }
    friend MontgomeryModInt32 operator*(const MontgomeryModInt32 &lhs,
                                        const MontgomeryModInt32 &rhs) {
        return MontgomeryModInt32(lhs) *= rhs;
    }
    friend MontgomeryModInt32 operator/(const MontgomeryModInt32 &lhs,
                                        const MontgomeryModInt32 &rhs) {
        return MontgomeryModInt32(lhs) /= rhs;
    }
    friend bool operator==(const MontgomeryModInt32 &lhs, const MontgomeryModInt32
&rhs) {
        return norm(lhs.v) == norm(rhs.v);
    }
    friend bool operator!=(const MontgomeryModInt32 &lhs, const MontgomeryModInt32
&rhs) {
        return norm(lhs.v) != norm(rhs.v);
    }
    friend std::istream &operator>>(std::istream &is, MontgomeryModInt32 &rhs) {
        return is >> rhs.v, rhs.v = reduce(u64(rhs.v) * r2), is;
    }
    friend std::ostream &operator<<(std::ostream &os, const MontgomeryModInt32 &rhs)
{
        return os << rhs.get();
    }
    constexpr MontgomeryModInt32 pow(i64 y) const {
        if ((y %= P - 1) < 0)
            y += P - 1; // phi(P) = P - 1, assume P is a prime number

        MontgomeryModInt32 res(1), x(*this);

        for (; y != 0; y >>= 1, x *= x)
            if (y & 1)
```

```
                res *= x;

        return res;
    }
};

template <std::uint32_t P> MontgomeryModInt32<P> sqrt(const MontgomeryModInt32<P>
&x) {
    using value_type = MontgomeryModInt32<P>;
    static constexpr value_type negtive_one(P - 1), ZERO(0);

    if (x == ZERO || x.pow(P - 1 >> 1) == negtive_one)
        return ZERO;

    if ((P & 3) == 3)
        return x.pow(P + 1 >> 2);

    static value_type w2, ax;
    ax = x;
    static std::random_device rd;
    static std::mt19937 gen(rd());
    static std::uniform_int_distribution<std::uint32_t> dis(1, P - 1);
    const value_type four(value_type(4) * x);
    static value_type t;

    do
        t = value_type(dis(gen)), w2 = t * t - four;

    while (w2.pow(P - 1 >> 1) != negtive_one);

    struct Field_P2 { // (A + Bx)(C+Dx)=(AC-BDa)+(AD+BC+BDt)x
    public:
        value_type a, b;
        Field_P2(const value_type &a, const value_type &b) : a(a), b(b) {}
        ~Field_P2() = default;
        Field_P2 &operator*=(const Field_P2 &rhs) {
            value_type tmp1(b * rhs.b), tmp2(a * rhs.a - tmp1 * ax),
                       tmp3(a * rhs.b + b * rhs.a + tmp1 * t);
            return a = tmp2, b = tmp3, *this;
        }
        Field_P2 pow(std::uint64_t y) const {
            Field_P2 res(value_type(1), ZERO), x(*this);

            for (; y != 0; y >>= 1, x *= x)
                if (y & 1)
                    res *= x;

            return res;
        }
    } res(ZERO, value_type(1));
    return res.pow(P + 1 >> 1).a;
}
```

```cpp
std::uint64_t get_len(std::uint64_t n) { // if n=0, boom
    return --n, n |= n >> 1, n |= n >> 2, n |= n >> 4, n |= n >> 8, n |= n >> 16, n
|= n >> 32, ++n;
}


template <std::uint32_t P> struct NTT {
public:
    using i32 = std::int32_t;
    using u32 = std::uint32_t;
    using i64 = std::int64_t;
    using u64 = std::uint64_t;
    using value_type = MontgomeryModInt32<P>;

private:
    static inline value_type ROOT[1 << 20], IROOT[1 << 20];

public:
    NTT() = delete;
    static void idft(i32 n, value_type x[]) {
        for (i32 i = 2; i < n; i <<= 1) {
            for (i32 j = 0, l = i >> 1; j != l; ++j) {
                value_type u = x[j], v = x[j + l];
                x[j] = u + v, x[j + l] = u - v;
            }

            for (i32 j = i, l = i >> 1, m = 1; j != n; j += i, ++m) {
                value_type root = IROOT[m];

                for (i32 k = 0; k != l; ++k) {
                    value_type u = x[j + k], v = x[j + k + l];
                    x[j + k] = u + v, x[j + k + l] = (u - v) * root;
                }
            }
        }

        value_type iv(P - (P - 1) / n);

        for (i32 j = 0, l = n >> 1; j != l; ++j) {
            value_type u = x[j] * iv, v = x[j + l] * iv;
            x[j] = u + v, x[j + l] = u - v;
        }
    }
    static void dft(i32 n, value_type x[]) {
        static i32 lim = 0;
        static constexpr u32 pr = value_type::get_pr();
        static_assert(pr != 0);
        static constexpr value_type G(pr);

        if (lim == 0) {
```

```
            ROOT[1 << 19] = G.pow(P - 1 >> 21), IROOT[1 << 19] = G.pow(-i32(P - 1 >>
21));

            for (i32 i = 18; i != -1; --i)
                ROOT[1 << i] = ROOT[1 << i + 1] * ROOT[1 << i + 1],
                               IROOT[1 << i] = IROOT[1 << i + 1] * IROOT[1 << i +
1];

            lim = 1;
        }

        while ((lim << 1) < n) {
            for (i32 i = lim + 1, e = lim << 1; i < e; ++i)
                ROOT[i] = ROOT[i - lim] * ROOT[lim], IROOT[i] = IROOT[i - lim] *
IROOT[lim];

            lim <<= 1;
        }

        for (i32 j = 0, l = n >> 1; j != l; ++j) {
            value_type u = x[j], v = x[j + l];
            x[j] = u + v, x[j + l] = u - v;
        }

        for (i32 i = n >> 1; i >= 2; i >>= 1) {
            for (i32 j = 0, l = i >> 1; j != l; ++j) {
                value_type u = x[j], v = x[j + l];
                x[j] = u + v, x[j + l] = u - v;
            }

            for (i32 j = i, l = i >> 1, m = 1; j != n; j += i, ++m) {
                value_type root = ROOT[m];

                for (i32 k = 0; k != l; ++k) {
                    value_type u = x[j + k], v = x[j + k + l] * root;
                    x[j + k] = u + v, x[j + k + l] = u - v;
                }
            }
        }
    }
};

template <std::uint32_t P> void dft(std::uint32_t n, MontgomeryModInt32<P> x[]) {
    NTT<P>::dft(n, x);
}

template <std::uint32_t P> void idft(std::uint32_t n, MontgomeryModInt32<P> x[]) {
    NTT<P>::idft(n, x);
}

template <std::uint32_t P>
std::vector<MontgomeryModInt32<P>> add(const std::vector<MontgomeryModInt32<P>> &x,
```

```
const std::vector<MontgomeryModInt32<P>> &y) {
    using i32 = std::int32_t;
    using value_type = MontgomeryModInt32<P>;
    std::vector<value_type> res(std::max(x.size(), y.size()));

    for (i32 i = 0, e = std::min(x.size(), y.size()); i != e; ++i)
        res[i] = x[i] + y[i];

    if (x.size() < y.size())
        std::copy(y.begin() + x.size(), y.end(), res.begin() + x.size());
    else
        std::copy(x.begin() + y.size(), x.end(), res.begin() + y.size());

    return res;
}

template <std::uint32_t P>
std::vector<MontgomeryModInt32<P>> sub(const std::vector<MontgomeryModInt32<P>> &x,
const std::vector<MontgomeryModInt32<P>> &y) {
    using i32 = std::int32_t;
    using value_type = MontgomeryModInt32<P>;
    std::vector<value_type> res(std::max(x.size(), y.size()));

    for (i32 i = 0, e = std::min(x.size(), y.size()); i != e; ++i)
        res[i] = x[i] - y[i];

    if (x.size() < y.size())
        for (i32 i = x.size(), e = y.size(); i != e; ++i)
            res[i] = -y[i];
    else
        std::copy(x.begin() + y.size(), x.end(), res.begin() + y.size());

    return res;
}

template <std::uint32_t P>
std::vector<MontgomeryModInt32<P>> mul(const std::vector<MontgomeryModInt32<P>> &x,
const std::vector<MontgomeryModInt32<P>> &y) {
    using i32 = std::int32_t;
    using value_type = MontgomeryModInt32<P>;
    static value_type a[1 << 21], b[1 << 21];
    std::vector<value_type> res(x.size() + y.size() - 1);

    if (std::min(x.size(), y.size()) <= 8) {
        i32 n = x.size(), m = y.size();
        std::fill(res.begin(), res.end(), 0);

        for (i32 i = 0; i != n; ++i)
            for (i32 j = 0; j != m; ++j)
                res[i + j] += x[i] * y[j];
    } else {
        std::copy(x.begin(), x.end(), a);
```

```
        std::copy(y.begin(), y.end(), b);
        i32 old_size = res.size(), len = get_len(old_size);
        std::fill(a + x.size(), a + len, 0);
        std::fill(b + y.size(), b + len, 0);
        dft(len, a), dft(len, b);
        res.resize(len);

        for (i32 i = 0; i != len; ++i)
            res[i] = a[i] * b[i];

        idft(len, res.data());
        res.resize(old_size);
    }

    return res;
}

template <std::uint32_t P>
std::vector<MontgomeryModInt32<P>> inv(std::vector<MontgomeryModInt32<P>> x) {
    assert(!x.empty());
    using value_type = MontgomeryModInt32<P>;
    using i32 = std::int32_t;
    static value_type a[1 << 21], b[1 << 21];
    i32 n = x.size(), len = get_len(n);
    x.resize(len);
    std::vector<value_type> res(len, 0);
    res[0] = x[0].inv();

    for (i32 i = 1; i != 32 && i < n; ++i) {
        value_type sum(0);

        for (i32 j = 1; j <= i; ++j)
            sum += x[j] * res[i - j];

        res[i] = -res[0] * sum;
    }

    for (i32 i = 64; i <= len; i <<= 1) {
        std::copy_n(x.begin(), i, a);
        std::copy_n(res.begin(), i, b);
        dft(i, a), dft(i, b);

        for (i32 j = 0; j != i; ++j)
            a[j] *= b[j];

        idft(i, a);
        std::fill_n(a, i >> 1, 0);
        dft(i, a);

        for (i32 j = 0; j != i; ++j)
            a[j] *= b[j];
```

```
        idft(i, a);

        for (i32 j = i >> 1; j != i; ++j)
            res[j] = -a[j];
    }

    return res.resize(n), res;
}

template <std::uint32_t P>
std::vector<MontgomeryModInt32<P>> quo(std::vector<MontgomeryModInt32<P>> x,
std::vector<MontgomeryModInt32<P>> y) {
    assert(!x.empty());
    assert(!y.empty());

    if (x.size() == 1)
        return {x[0] *y[0].inv()};

    using value_type = MontgomeryModInt32<P>;

    using i32 = std::int32_t;

    static value_type a[1 << 21], b[1 << 21], c[1 << 21];

    i32 n = x.size(), len = get_len(n);

    x.resize(len), y.resize(len, 0);

    std::vector<value_type> g0 = inv(std::vector<value_type>(y.begin(), y.begin() +
(len >> 1))),
                             res(n);

    std::copy_n(g0.begin(), len >> 1, a); // a->g0

    std::copy_n(x.begin(), len >> 1, b);

    std::fill_n(a + (len >> 1), len >> 1, 0);

    std::fill_n(b + (len >> 1), len >> 1, 0);

    dft(len, a), dft(len, b);

    for (i32 i = 0; i != len; ++i)
        b[i] *= a[i];

    idft(len, b);
    std::fill_n(b + (len >> 1), len >> 1, 0); // b->q0
    std::copy_n(b, len >> 1, res.begin());    // res->q0
    std::copy_n(y.begin(), len, c);           // c->y
    dft(len, b), dft(len, c);

    for (i32 i = 0; i != len; ++i)
```

```
        c[i] *= b[i];

    idft(len, c);
    std::fill_n(c, len >> 1, 0);

    for (i32 i = len >> 1; i != len; ++i)
        c[i] = x[i] - c[i];

    dft(len, c);

    for (i32 i = 0; i != len; ++i)
        c[i] *= a[i];

    idft(len, c);
    std::copy(c + (len >> 1), c + n, res.begin() + (len >> 1));
    return res;
}

template <std::uint32_t P>
std::vector<MontgomeryModInt32<P>> quorem(std::vector<MontgomeryModInt32<P>> x,
std::vector<MontgomeryModInt32<P>> y) {
    using i32 = std::int32_t;
    using value_type = MontgomeryModInt32<P>;

    if (x.size() < y.size())
        return {0};

    i32 n = x.size() - y.size() + 1;

    std::reverse(x.begin(), x.end());

    std::reverse(y.begin(), y.end());

    x.resize(n);

    std::vector<value_type> res = quo(x, y);

    std::reverse(res.begin(), res.end());

    return res;
}

template <std::uint32_t P>
std::pair<std::vector<MontgomeryModInt32<P>>, std::vector<MontgomeryModInt32<P>>>
quo_with_rem(
            const std::vector<MontgomeryModInt32<P>> &x,
const std::vector<MontgomeryModInt32<P>> &y) {
    using i32 = std::int32_t;
    using value_type = MontgomeryModInt32<P>;
    static value_type a[1 << 21], b[1 << 21];

    if (x.size() < y.size())
```

```
        return {{0}, x};

    std::vector<value_type> q = quorem(x, y);

    if (y.size() == 1)
        return {q, {0}};

    i32 len = get_len(std::max(q.size(), y.size())), overlap = q.size() + y.size() -
1 - len;

    std::copy_n(q.begin(), q.size(), a);

    std::copy_n(y.begin(), y.size(), b);

    std::fill(a + q.size(), a + len, 0);

    std::fill(b + y.size(), b + len, 0);

    dft(len, a), dft(len, b);

    for (i32 i = 0; i != len; ++i)
        a[i] *= b[i];

    idft(len, a);
    std::vector<value_type> rem(y.size() - 1);

    if (overlap > 0)
        for (i32 i = 0, e = rem.size(); i != overlap && i != e; ++i)
            a[i] -= x[len + i];

    for (i32 i = 0, e = rem.size(); i != e; ++i)
        rem[i] = x[i] - a[i];

    return {q, rem};
}

template <std::uint32_t P>
std::vector<MontgomeryModInt32<P>> rem(const std::vector<MontgomeryModInt32<P>> &x,
const std::vector<MontgomeryModInt32<P>> &y) {
    return quo_with_rem(x, y).second;
}

template <std::uint32_t P>
std::vector<MontgomeryModInt32<P>> deriv(const std::vector<MontgomeryModInt32<P>>
&x) {
    using i32 = std::int32_t;
    using value_type = MontgomeryModInt32<P>;

    if (x.size() == 1)
        return {0};

    std::vector<value_type> res(x.size() - 1);
```

```
    for (i32 i = 1, e = x.size(); i != e; ++i)
        res[i - 1] = x[i] * value_type(i);

    return res;
}

template <std::uint32_t P>
std::vector<MontgomeryModInt32<P>> integr(const std::vector<MontgomeryModInt32<P>>
&x) {
    using i32 = std::int32_t;
    using value_type = MontgomeryModInt32<P>;
    static i32 lim = 0;
    static value_type INV[1 << 21];
    i32 len = x.size() + 1;
    std::vector<value_type> res(len);
    res[0] = 0;

    if (lim < len) {
        INV[1] = 1;

        for (i32 i = (lim < 2 ? 2 : lim); i < len; ++i)
            INV[i] = value_type(P - P / i) * INV[P % i];

        lim = len;
    }

    for (i32 i = 1; i != len; ++i)
        res[i] = x[i - 1] * INV[i];

    return res;
}

template <std::uint32_t P>
std::vector<MontgomeryModInt32<P>> log(const std::vector<MontgomeryModInt32<P>> &x)
{
    return x.size() == 1 ? std::vector<MontgomeryModInt32<P>> {0} :
           integr(quo(deriv(x), x));
}

template <std::uint32_t P>
std::vector<MontgomeryModInt32<P>> exp(std::vector<MontgomeryModInt32<P>> x) {
    assert(x[0] == 0);
    using i32 = std::int32_t;
    using value_type = MontgomeryModInt32<P>;
    i32 n = x.size(), len = get_len(n);

    if (n == 1)
        return {1};

    static value_type a[1 << 21], b[1 << 21], c[1 << 21], d[1 << 21], e[1 << 21],
f[1 << 21],
```

```
        INV[1 << 21];

    static i32 lim = 0;

    if (lim < len) {
        INV[1] = 1;

        for (i32 i = (lim < 2 ? 2 : lim); i < len; ++i)
            INV[i] = value_type(P - P / i) * INV[P % i];

        lim = len;
    }

    static constexpr value_type ZERO(0);
    x.resize(len);
    std::vector<value_type> res(len);
    res[0] = 1, res[1] = x[1];
    std::fill_n(a, len, 0);
    std::fill_n(c, len, 0);
    a[0] = 1, a[1] = -res[1];                                   // a->h0

    for (i32 i = 1; i != len; ++i)
        b[i - 1] = value_type(i) * x[i]; // b->f'

    std::copy_n(res.begin(), 2, c);                             // c->g0
    dft(4, c);                                                  // c-> dft g0
    f[0] = res[1];

    for (i32 i = 4; i <= len; i <<= 1) {
        for (i32 j = i >> 2; j != i >> 1; ++j)
            f[j - 1] = res[j] * value_type(j);

        std::copy_n(b, (i >> 1) - 1, d); // d->f0'
        std::fill(d + (i >> 1) - 1, d + i, ZERO);
        dft(i >> 1, d);

        for (i32 j = 0; j != i >> 1; ++j)
            d[j] *= c[j];

        idft(i >> 1, d); // d->g0f0'

        for (i32 j = 0; j != (i >> 1) - 2; ++j)
            d[j + (i >> 1)] = f[j] - d[j], d[j] = ZERO;

        d[(i >> 1) - 2] -= f[(i >> 1) - 2];
        d[(i >> 1) - 1] = -d[(i >> 1) - 1];
        std::copy_n(a, i, e); // e->h0
        dft(i, e), dft(i, d);

        for (i32 j = 0; j != i; ++j)
            d[j] *= e[j];
```

```
        idft(i, d); // d->h0(g0'-g0f0')
        std::vector<value_type> it(i);

        for (i32 j = i >> 1; j != i; ++j)
            it[j] = d[j - 1] * INV[j] - x[j];

        std::fill_n(it.begin(), i >> 1, ZERO);
        dft(i, it.data());

        for (i32 j = 0; j != i; ++j)
            c[j] *= it[j];

        idft(i, c);

        for (i32 j = i >> 1; j != i; ++j)
            res[j] = -c[j];

        if (i != len) {
            std::copy_n(res.begin(), i, c);
            dft(i << 1, c);

            for (i32 j = 0; j != i; ++j)
                d[j] = c[j] * e[j];

            idft(i, d);
            std::fill_n(d, i >> 1, ZERO);
            dft(i, d);

            for (i32 j = 0; j != i; ++j)
                d[j] *= e[j];

            idft(i, d);

            for (i32 j = i >> 1; j != i; ++j)
                a[j] = -d[j];
        }
    }

    return res.resize(n), res;
}

template <std::uint32_t P>
std::vector<MontgomeryModInt32<P>> sqrt(std::vector<MontgomeryModInt32<P>> x) {
    using i32 = std::int32_t;
    using value_type = MontgomeryModInt32<P>;
    static value_type a[1 << 21], b[1 << 21], c[1 << 21], d[1 << 21];
    static constexpr value_type iv2(P - (P - 1 >> 1)), ZERO(0);
    i32 n = x.size(), len = get_len(n);
    x.resize(len);
    std::vector<value_type> res(len);
    std::uint32_t k = std::uint32_t(sqrt(x[0]));
    assert(k != 0);
```

```
    res[0] = std::min(P - k, k);
    std::fill_n(a, len, 0);
    a[0] = res[0].inv();        // a->h0
    b[0] = res[0], b[1] = ZERO; // b->g0
    dft(2, b);                  // b-> dft g0

    for (i32 i = 2; i <= len; i <<= 1) {
        for (i32 j = 0; j != i >> 1; ++j)
            c[j] = b[j] * b[j];

        idft(i >> 1, c);

        for (i32 j = 0; j != (i >> 1) - 1; ++j)
            c[j + (i >> 1)] = c[j] - x[j] - x[j + (i >> 1)], c[j] = ZERO;

        c[(i >> 1) - 1] -= x[(i >> 1) - 1];
        c[i - 1] -= x[i - 1];
        std::copy_n(a, i, d); // d->h0
        dft(i, d), dft(i, c);

        for (i32 j = 0; j != i; ++j)
            c[j] *= d[j];

        idft(i, c);

        for (i32 j = i >> 1; j != i; ++j)
            res[j] = -c[j] * iv2;

        if (i != len) {
            std::copy_n(res.begin(), i, b);
            dft(i, b); // b-> dft g0

            for (i32 j = 0; j != i; ++j)
                c[j] = b[j] * d[j];

            idft(i, c);
            std::fill_n(c, i >> 1, ZERO);
            dft(i, c);

            for (i32 j = 0; j != i; ++j)
                c[j] *= d[j];

            idft(i, c);

            for (i32 j = i >> 1; j != i; ++j)
                a[j] = -c[j];
        }
    }

    return res.resize(n), res;
}
```

```cpp
template <std::uint32_t P>
std::vector<MontgomeryModInt32<P>> sqrtinv(std::vector<MontgomeryModInt32<P>> x) {
    using i32 = std::int32_t;
    using value_type = MontgomeryModInt32<P>;
    static value_type a[1 << 21], b[1 << 21], c[1 << 21], d[1 << 21];
    static constexpr value_type iv2(P - (P - 1 >> 1)), ZERO(0);
    i32 n = x.size(), len = get_len(n);
    x.resize(len);
    std::vector<value_type> res(len);
    std::uint32_t k = std::uint32_t(sqrt(x[0]));
    assert(k != 0);
    res[0] = std::min(P - k, k);
    std::fill_n(a, len, 0);
    a[0] = res[0].inv();        // a->h0
    b[0] = res[0], b[1] = ZERO; // b->g0
    dft(2, b);                  // b-> dft g0

    for (i32 i = 2; i <= len; i <<= 1) {
        for (i32 j = 0; j != i >> 1; ++j)
            c[j] = b[j] * b[j];

        idft(i >> 1, c);

        for (i32 j = 0; j != (i >> 1) - 1; ++j)
            c[j + (i >> 1)] = c[j] - x[j] - x[j + (i >> 1)], c[j] = ZERO;

        c[(i >> 1) - 1] -= x[(i >> 1) - 1];
        c[i - 1] -= x[i - 1];
        std::copy_n(a, i, d); // d->h0
        dft(i, d), dft(i, c);

        for (i32 j = 0; j != i; ++j)
            c[j] *= d[j];

        idft(i, c);

        for (i32 j = i >> 1; j != i; ++j)
            res[j] = -c[j] * iv2;

        std::copy_n(res.begin(), i, b);
        dft(i, b); // b-> dft g0

        for (i32 j = 0; j != i; ++j)
            c[j] = b[j] * d[j];

        idft(i, c);
        std::fill_n(c, i >> 1, ZERO);
        dft(i, c);

        for (i32 j = 0; j != i; ++j)
            c[j] *= d[j];
```

```
        idft(i, c);

        for (i32 j = i >> 1; j != i; ++j)
            a[j] = -c[j];
    }

    return std::vector<MontgomeryModInt32<P>>(a, a + n);
}

template <std::uint32_t P>
std::vector<MontgomeryModInt32<P>> pow(const std::vector<MontgomeryModInt32<P>> &x,
const MontgomeryModInt32<P> &k) {
    std::vector<MontgomeryModInt32<P>> y(log(x));

    for (auto &i : y)
        i *= k;

    return exp(y);
}

template <std::uint32_t P>
std::vector<MontgomeryModInt32<P>> operator+(const
std::vector<MontgomeryModInt32<P>> &x,
const std::vector<MontgomeryModInt32<P>> &y) {
    return add(x, y);
}

template <std::uint32_t P>
std::vector<MontgomeryModInt32<P>> operator-(const
std::vector<MontgomeryModInt32<P>> &x,
const std::vector<MontgomeryModInt32<P>> &y) {
    return sub(x, y);
}

template <std::uint32_t P>
std::vector<MontgomeryModInt32<P>> operator*(const
std::vector<MontgomeryModInt32<P>> &x,
const std::vector<MontgomeryModInt32<P>> &y) {
    return mul(x, y);
}

template <std::uint32_t P>
std::vector<MontgomeryModInt32<P>> operator/(const
std::vector<MontgomeryModInt32<P>> &x,
const std::vector<MontgomeryModInt32<P>> &y) {
    return quorem(x, y);
}

template <std::uint32_t P>
std::vector<MontgomeryModInt32<P>> operator%(const
std::vector<MontgomeryModInt32<P>> &x,
const std::vector<MontgomeryModInt32<P>> &y) {
```

```
    return rem(x, y);
}
```